

Checkpoint Feedback Rescope Resolutions

Async Checkpoint Feedback — Rescope Resolutions

Prepared: 2026-06-01 · **For:** Hume **Purpose:** Settle the seven follow-up notes on the rescope, grounded in the actual code rather than assertion. Companion to `CHECKPOINT_FEEDBACK_ASSESSMENT_2026-06-01.md`.

1. The `feedback_loop.py` constants — verified from source

The whole rescope leans on three claims about `feedback_loop.py`. Settled verbatim from the file:

```
PRIOR_STRENGTH = 50      # pseudo-observations at the 50% prior
PRIOR_ACCURACY = 0.5     # uninformative prior
MIN_SIGNALS_FOR_ADJUSTMENT = 10
ACCURACY_BLEND = 0.4     # how aggressively shrunk accuracy moves weights
```

The blending math, verbatim:

```
if n < MIN_SIGNALS_FOR_ADJUSTMENT:      # hard skip
    continue
shrunk_accuracy = (k + PRIOR_STRENGTH * PRIOR_ACCURACY) / (n + PRIOR_STRENGTH)
multiplier = 0.6 + (shrunk_accuracy * 0.8)
multiplier = max(0.5, min(1.5, multiplier)) # clamp
weights[factor] = old_w*(1-ACCURACY_BLEND) + new_w*ACCURACY_BLEND # 0.4 blend
```

Precision correction to my earlier wording: `n < 10` is a **hard** `continue`, not a "soft skip" — I undersold that. But its *effect* is immaterial, and the code's own comment says so: At `n=10`: `data weight = 10/60 ≈ 17%`. So at the gate the posterior is `~83%` prior, the `multiplier ≈ 1.0`, and any movement above the gate is further damped by `ACCURACY_BLEND = 0.4` and clamped to `[0.5, 1.5]`.

Net, settled:

- The threshold is **10, not 20**.
 - Below 10, adjustment is skipped — but would be `~nothing` anyway (`≈17%` data weight).
 - Above 10, the real mechanism is the continuous beta-binomial shrinkage, not a gate.
 - The only `20` in the file is a comment (the `old MIN_SIGNALS_FOR_ADJUSTMENT = 20`).
- There is no live $N \geq 20$ weight threshold anywhere.**

This is now verified, not asserted.

2. "Blocking" — agreed, and the motivation stays

Generation never waits on settlement, so "blocking" was the wrong word. But the actual complaint stands and the rescope keeps it central: **learning cannot begin pre-settlement, and nothing is sealed at decision time with the right fields.** That is a real latency-of-*learning* problem, and it is exactly why we seal `system_version` + `reasoning_fingerprint` now. Only the word changes; the motivation is not dropped.

3. Guard against accidental in-place edits

`prediction_log` currently has only an RLS policy `FOR ALL USING (true)` — no trigger, no append-only rule. So today nothing structurally prevents an accidental in-place edit; it is pure discipline.

The cheap real fix — which is what `sealed_hash` was actually reaching for (catching *your own* accidental mutation, not an adversary) — is a one-time migration. **Recommended (Option A): block UPDATE/DELETE at the policy level.**

```
-- Option A: append-only via RLS (keep INSERT + SELECT, deny UPDATE/DELETE)
DROP POLICY prediction_log_all ON prediction_log;
CREATE POLICY pl_insert ON prediction_log FOR INSERT WITH CHECK (true);
CREATE POLICY pl_select ON prediction_log FOR SELECT USING (true);
-- (no UPDATE/DELETE policy = denied)
```

Alternative (Option B): a `BEFORE UPDATE` trigger that `RAISE EXCEPTION`. Either makes the table genuinely append-only and catches a fat-finger — the honest version of the hash's intent, without the ceremony.

4. The `version_cohort_break` trigger — precise and automatic

My earlier three-item list ("analyst scoring math, Socratic model roles, corpus-callosum logic") was illustrative. The precise, automatable rule uses data the system **already captures**: `socratic_analyses.prompt_versions` (a jsonb of `model_a/b/c` + `corpus_callosum` versions) is written on every run.

```
cohort_key = hash( prompt_versions[model_a, model_b, model_c, corpus_callosum]
                  + git_short_hash(analyst.py, target_engine.py) )

version_cohort_break = (cohort_key != previous_prediction.cohort_key)
```

The seal hook computes this at write time — fully derived, zero manual flagging, so it does **not** depend on you remembering. Frontend/discovery churn touches none of those inputs, so it never trips a break (matching intent).

Open decision for you: is `analyst.py` + `target_engine.py` the right "judgment file set" to watch, or would you add/remove one?

5. `reasoning_fingerprint` — ~80% free, one real gap

From the row `run_socratic.py` actually writes:

Free from existing `socratic_analyses` jsonb (an extractor, no change to the analysis path):

- `unresolved_questions` = the `disagreements` array (the judgment-type ones the corpus callosum didn't resolve).
- per-model `confidence` = already inside each `model_a/b/c` jsonb.
- `dissenting_model` = computable by comparing the three model verdicts / confidences.

The gap is `conviction`. In socratic mode the row writes `final_verdict: None` — overall conviction is deliberately left for the human judgment card later, so there is no single conviction value at seal time. Two honest options:

- **(a)** seal a *computed proxy* (e.g. agreement × mean model confidence) — still no change to the generation path; or
- **(b)** leave `conviction` null at seal and let the judgment card backfill it when you decide.

So P1 is one `ALTER` + an extractor reading existing jsonb — not a change to the model prompts — provided you accept (a) or (b) for conviction. It stays a one-day item.

6. The cohort sensitivity check — written as a rule

It must not run as a vague habit. The rule:

- **Precondition / cadence:** run it only once at least **two cohorts each have $N \geq 10$** graded outcomes. Before that the comparison is pure noise — so for now it simply does not run.
- **Test for material divergence:** directional hit-rate (or median band-error) differing by **$\geq 20\text{pp}$ with $N \geq 10$ on both sides** of the break.
- **Action if yes:** pooling across that break is no longer valid for the **Corrective** tier — demote pre-break cohorts to reference-only and surface it to you.
- **Action if no:** keep pooling.

Pooling is the standing default; it only stops at a break that empirically fails the test, and never before the data can support the test.

7. Interim value before $N \geq 10$ — the plain answer

For **learning/calibration: log quietly and wait.** Nothing actionable exists before $\sim N \geq 10$ (realistically Q4). Anything that pretends to be a "learned pattern" at $N < 10$ should be suppressed, not surfaced.

The **one** honest exception is not calibration and needs no N: the **per-position breach / kill-condition monitor** (the fast clock). A single kill-condition trip is actionable on its own merits — you act on the position, not on a statistical pattern — and the system **already does this** via `kill_condition_eval.py` against your defined `kill_triggers`. That is real interim value and it is N-independent. Anything beyond that before $N \geq 10$ would be invented value, so none is proposed.

What this leaves to build (rescoped P0/P1)

#	Item	Nature	Size
P0	Resolve <code>thesis_outcomes</code> name collision (introspect → new name or extend <code>prediction_outcomes</code>)	migration	~1 h
P0	Fix <code>analysis_id</code> FK type → <code>bigint</code> (matches <code>socratic_analyses</code> PK)	migration	~0.5 h
P0	Date-pin grading + store the exact date used	code	~2 h
P1	<code>ALTER prediction_log add system_version, reasoning_fingerprint</code> ; populate version from git hash	migration + extractor	~2 h
P1	Append-only guard on <code>prediction_log</code> (Option A RLS)	migration	~0.5 h
P1	Derived <code>cohort_key</code> / <code>version_cohort_break</code> at seal; Corrective at $N \geq 10$	code	~½ day
P1	<code>reasoning_fingerprint</code> extractor over existing <code>socratic_analyses</code> jsonb (conviction via proxy or judgment-card backfill)	code	included above

Decisions still owed by you: the judgment file-set for the cohort key (§4); conviction option (a) or (b) (§5).

Evidence base: `scripts/feedback_loop.py` (constants + blend), `scripts/run_socratic.py` (sealed row contents), `supabase/2026-04-26_prediction_log.sql` (RLS policy), `supabase/2026-05-15_socratic.sql` (schema, `bigint` PK, `prompt_versions`, disagreements), `scripts/kill_condition_eval.py`.